第07堂｜Vibe Coding 驅動：智慧動態報表系統開發

學習目標

完成本堂課後，學員將能夠：

- 運用 Vibe Coding 實現從資料取得到報表生成的**端到端（End-to-End）**流程
- 整合外部公開 API，取得即時資料（匯率、股價、天氣等）
- 建立動態資料模型，讓報表自動隨資料更新
- 輸出格式多樣的報表（Excel、HTML、PDF）

核心概念說明

1. 什麼是「端到端報表系統」？

端到端（End-to-End）意味著整個流程從頭到尾完全自動化，不需要人工介入：

```
外部資料來源（API / 資料庫 / 檔案）
    │
    ▼ 自動擷取資料
原始資料
    │
    ▼ 資料清理與計算
處理後的數據
    │
    ▼ 套用報表模板
格式化報表
    │
    ▼ 自動輸出
Excel / PDF / HTML / 郵件附件
```

傳統 vs. 智慧動態報表的差異：

	傳統手工報表	智慧動態報表系統
資料來源	手動下載、複製貼上	自動從 API 或資料庫取得
更新頻率	需要人工觸發	可排程定時自動執行
人力需求	每次都需要人力介入	設置一次，持續自動運行
錯誤率	人工操作容易出錯	流程一致，錯誤可追蹤

---

2. API 的基礎概念

API（Application Programming Interface，應用程式介面）是讓不同軟體之間互相溝通的橋樑。

一個生活化的比喻：

在餐廳點餐時：

- **你** = 你的程式
- **菜單** = API 文件（說明可以點什麼、怎麼點）
- **服務生** = API
- **廚房** = 提供資料的服務

你不需要知道廚房怎麼運作，只需要按照菜單（API 文件）點餐（發送請求），服務生（API）就會把餐點（資料）送來。

REST API 的基本結構：

```
請求（Request）：
    方式: GET / POST / PUT / DELETE
    網址: https://api.example.com/data?param=value
    標頭: Authorization: Bearer YOUR_API_KEY
    主體: { "key": "value" }  ← POST 請求才有

回應（Response）：
    狀態碼: 200 成功 / 401 未授權 / 404 找不到 / 500 伺服器錯誤
    資料: { "result": [...] }  ← 通常是 JSON 格式
```

常用的公開 API（適合新手練習）：

API 名稱	提供的資料	是否需要 Key
ExchangeRate-API	匯率資料	是（免費）
OpenWeatherMap	天氣資料	是（免費）
CoinGecko	加密貨幣價格	否（部分免費）
REST Countries	各國資訊	否
政府資料開放平台	台灣各類公開資料	否

---

3. 動態資料建模

靜態報表：資料是固定的，報表也是固定的。 **動態報表**：資料隨時間變化，報表自動根據最新資料計算和呈現。

動態報表的三個核心要素：

1. **資料層**：從哪裡取得資料？取得頻率？
2. **計算層**：需要做哪些計算（加總、平均、同比增長）？
3. **呈現層**：如何顯示（表格、圖表、關鍵指標）？

一個業務日報表的動態模型：

```
計算「今日業績」：
    今日銷售 = 從資料庫取得今日所有訂單金額
    目標業績 = 從設定檔讀取本月目標 / 本月天數
    達成率 = 今日銷售 / 目標業績 * 100%
    與昨日比較 = (今日銷售 - 昨日銷售) / 昨日銷售 * 100%

呈現方式：
    KPI 卡片：今日業績 / 達成率（顏色：達成=綠色，未達成=紅色）
    長條圖：本月每日業績趨勢
    排行榜：今日業務員業績排名 Top 10
```

---

4. 報表輸出格式選擇

格式	工具	適合情境
Excel (.xlsx)	openpyxl、xlsxwriter	給需要進一步分析資料的人
HTML	Jinja2 + matplotlib	在瀏覽器或郵件中直接檢視
PDF	reportlab、weasyprint	正式文件、合約、對外報告
PowerPoint (.pptx)	python-pptx	管理報告、簡報用途

---

5. Vibe Coding 在複雜系統中的應用策略

當需要構建一個複雜的系統（如完整的報表生成系統），Vibe Coding 的策略是：

「分而治之」原則： 不要一次把整個系統描述給 AI，而是**拆解成小模組**，逐一生成和測試，再組合在一起。

```
複雜需求：「一個自動產生每日業務報表的系統」

拆解為：
  模組 1：資料取得函式（呼叫 API）
  模組 2：資料清理函式（處理缺失值、格式化）
  模組 3：指標計算函式（加總、排名、增長率）
  模組 4：報表樣式函式（設定 Excel 格式、圖表）
  模組 5：主程式（組合以上模組，設定排程）

每個模組先單獨開發和測試，最後整合。
```

實作步驟（逐步教學）

實作一：呼叫公開 API 取得即時資料

步驟 1：向 GitHub Copilot 描述第一個子模組

```
請幫我寫一個 Python 函式，呼叫 ExchangeRate-API 取得即時匯率：
1. 函式名稱：get_exchange_rates(base_currency="TWD")
2. API Key 從環境變數讀取
3. 回傳一個字典，包含主要貨幣對台幣的匯率（USD、EUR、JPY、CNY、HKD）
4. 加入錯誤處理：API 呼叫失敗時回傳 None 並記錄錯誤
5. 加入簡單的快取機制：如果 1 小時內已經呼叫過，回傳快取結果
```

步驟 2：測試這個函式

1. 直接呼叫函式，確認能正確取得匯率資料
2. 設定錯誤的 API Key，確認錯誤處理正常運作

實作二：建立 Excel 動態報表

步驟 2：向 Copilot 描述報表生成模組

```
請幫我寫一個 Excel 報表生成函式，輸入是匯率資料字典，輸出是一份格式化的 Excel 報表：
1. 第一個工作表「匯率快照」：
   - 顯示今日日期
   - 每個貨幣對一列，顯示：幣別、匯率、與上次比較的變化（漲/跌）
   - USD、EUR 欄加上適當格式（2位小數）、JPY 用0位小數
   - 若匯率比前次上漲，字體設為紅色；若下跌，設為綠色
2. 第二個工作表「圖表」：
   - 插入一個長條圖，顯示各幣別換算 NT$100 可得多少外幣
3. 報表儲存檔名格式：「匯率報表_YYYYMMDD.xlsx」
```

實作三：整合為完整的端到端流程

步驟 1：向 Copilot 描述主程式邏輯

```
請幫我整合剛才開發的函式，建立一個完整的自動化報表系統：
1. 主程式執行時：
   a. 從多個 API 取得資料（匯率 + 今日天氣，兩個資料都要錯誤處理）
   b. 生成 Excel 報表
   c. 生成一份 HTML 摘要（只有關鍵指標，容易用郵件傳送）
   d. 顯示「報表生成成功！」和檔案路徑
2. 加入執行時間記錄：顯示每個步驟花了多少秒
3. 所有設定（API Key、輸出資料夾）都從環境變數讀取
```

程式碼範例（虛擬碼）

API 資料取得函式

```
載入 requests, json, datetime, os 套件

快取 = {}  # 存放暫時的快取資料

函式 取得匯率(基準貨幣="TWD"):
    # 檢查快取
    快取鍵 = f"匯率_{基準貨幣}"
    如果 快取鍵 在 快取 中:
        快取時間 = 快取[快取鍵]["取得時間"]
        如果 距今不超過 60 分鐘:
            回傳 快取[快取鍵]["資料"]

    # 呼叫 API
    API_KEY = 從環境變數讀取("EXCHANGE_RATE_API_KEY")
    API_URL = f"https://v6.exchangerate-api.com/v6/{API_KEY}/latest/{基準貨幣}"

    嘗試:
        回應 = requests.get(API_URL, 逾時=10秒)
        回應.確認狀態碼成功()
        資料 = 解析JSON(回應)
        結果 = {
            "USD": 資料["conversion_rates"]["USD"],
            "EUR": 資料["conversion_rates"]["EUR"],
            "JPY": 資料["conversion_rates"]["JPY"],
            # ...
        }
        快取[快取鍵] = {"資料": 結果, "取得時間": 現在時間()}
        回傳 結果

    如果發生 連線錯誤:
        記錄錯誤("API 連線失敗")
        回傳 None
```

Excel 動態報表生成

```
載入 openpyxl 套件
載入 openpyxl.styles（用於設定格式）

函式 生成匯率報表(今日匯率, 昨日匯率=None):
    活頁簿 = 建立新的Excel活頁簿()
    工作表 = 活頁簿.取得活動工作表()
    工作表.名稱 = "匯率快照"

    # 寫入標題列
    標題 = ["幣別", "今日匯率", "變動", "變動方向"]
    工作表.寫入第一列(標題)
    設定標題列格式(粗體, 背景色=灰色)

    # 寫入資料
    對 幣別, 匯率 在 今日匯率.items():
```

```
        變動 = 計算變動(今日匯率[幣別], 昨日匯率[幣別]) if 昨日匯率 else 0
        方向 = "▲" if 變動 > 0 else "▼" if 變動 < 0 else "-"
        顏色 = 紅色 if 變動 > 0 else 綠色 if 變動 < 0 else 黑色

        新列 = [幣別, 匯率, 變動, 方向]
        工作表.加入新列(新列)
        設定最後列字體顏色(顏色)

    # 插入圖表
    圖表 = 建立長條圖()
    圖表.資料來源 = 工作表["B2":"B末行"]
    圖表.類別資料 = 工作表["A2":"A末行"]
    圖表.標題 = "各幣別今日匯率"
    工作表插入圖表(圖表, 位置="E2")

    # 儲存
    今日日期 = 取得今日日期字串()
    活頁簿.儲存(f"匯率報表_{今日日期}.xlsx")
    顯示 "報表已儲存"
```

練習題 / 作業

課堂練習

練習 1：API 使用場景分析

思考你的工作場景，找出 2-3 個可以「透過 API 自動取得資料」的需求：

需求描述	可能可用的 API	資料更新頻率

練習 2：報表設計

設計一份你工作中需要的「動態報表」：

- 報表的主題是什麼？
- 資料來源有哪些（手動填寫 / 資料庫 / API）？
- 需要計算哪些指標？
- 希望呈現什麼圖表？
- 報表的讀者是誰？需要什麼格式？

課後作業

1. **API 實作**：選一個公開 API（建議從「政府資料開放平台」找有興趣的資料集），用 GitHub Copilot 協助你撰寫一個能取得並顯示資料的程式。

2. **報表生成**：在 API 資料取得成功後，嘗試將資料輸出成一份 Excel 報表（可以只有資料，不需要太複雜的格式）。

延伸閱讀

API 學習資源：

- requests 套件文件 — https://requests.readthedocs.io/
- 台灣政府資料開放平台 — https://data.gov.tw/
- Postman（API 測試工具）— https://www.postman.com/

報表生成工具：

- openpyxl 文件 — https://openpyxl.readthedocs.io/
- xlsxwriter（另一個 Excel 工具，圖表功能更強）— https://xlsxwriter.readthedocs.io/
- Jinja2（HTML 模板引擎）— https://jinja.palletsprojects.com/

資料視覺化：

- matplotlib — https://matplotlib.org/
- plotly（互動式圖表）— https://plotly.com/python/